This is one of a series of Colab notebooks created for the **IR course**. It demonstrates how we can index a collection, and how to access an index to highlight some index analysis.

The **learning outcomes** of the this notebook are:

- PyTerrier setup.
- Preprocessing.
- Indexing a collection.
- Accessing and exploring the index.

What is PyTerrier?

PyTerrier is a Python framework, but uses the underlying Terrier information retrieval toolkit for many indexing and retrieval operations. Terrier is written in Java and has a long history dating back to 2001. PyTerrier makes it easy to perform IR experiments in Python, but using the mature Terrier platform for the expensive indexing and retrieval operations.

Setup

We will first install Pyterrier as follows:

```
#install the Pyterrier framework
#!pip install python-terrier
```

The next step is to initialise PyTerrier. This is performed using PyTerrier's init() method. The init() method is needed as PyTerrier must download Terrier's jar file and start the Java virtual machine. We prevent init() from being called more than once by checking started().

```
import pyterrier as pt

if not pt.started():
  pt.init()
```

```
C:\Users\Mazen\AppData\Local\Temp\ipykernel_17956\2060694631.py:3: DeprecationWarning: Call to deprecated function (o
  if not pt.started():
Java started and loaded: pyterrier.java.colab, pyterrier.java, pyterrier.terrier.java [version=5.11 (build: craig.mac
C:\Users\Mazen\AppData\Local\Temp\ipykernel_17956\2060694631.py:4: DeprecationWarning: Call to deprecated method pt.i
java is now started automatically with default settings. To force initialisation early, run:
pt.java.init() # optional, forces java initialisation
  pt.init()
```

We will import all the python libraries needed for this lab

```
#we need to import the following libraries.
import pandas as pd
import re
import os

import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Mazen\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Mazen\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Mazen\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

Data preparation

We will first create five textual documents.

```
docs_df = pd.DataFrame([["d0", "breakthrough drug for schizophrenia"],
                        ["d1", "new schizophrenia drug"],
                        ["d2", "new approach for treatment of schizophrenia"],
                        ["d3", "new hopes for schizophrenia patients"] ],
                        columns=["docno", "raw_text"])

docs_df
```

	docno	raw_text
0	d0	breakthrough drug for schizophrenia
1	d1	new schizophrenia drug
2	d2	new approach for treatment of schizophrenia
3	d3	new hopes for schizophrenia patients

Before indexing our data we need to do the following processing steps:

1. **Remove stopwords.**
2. **Stemming.**

```
def preprocess_text(text):
    stemmer = PorterStemmer()
    stop_words = set(stopwords.words('english'))

    text = text.lower()
    tokens = word_tokenize(text)
    tokens = [token for token in tokens if token.isalnum() and token not in stop_words]
    tokens = [stemmer.stem(token) for token in tokens]
    result = ' '.join(tokens)

    return result
```

```
# Apply the preprocess function on each document
docs_df['processed_text'] = docs_df['raw_text'].apply(preprocess_text)
print(docs_df[['docno', 'raw_text', 'processed_text']])
```

```
  docno                                     raw_text  \
0    d0          breakthrough drug for schizophrenia
1    d1                       new schizophrenia drug
2    d2  new approach for treatment of schizophrenia
3    d3         new hopes for schizophrenia patients

                         processed_text
0       breakthrough drug schizophrenia
1                new schizophrenia drug
2  new approach treatment schizophrenia
3        new hope schizophrenia patient
```

Next, we will index the dataframe's documents. The index, with all its data structures, is saved into a directory called **myFirstIndex**.

```
current_dir = os.getcwd()
index_path = os.path.join(current_dir, "invertedIndex")

# Create an indexer for English text
indexer = pt.DFIndexer(index_path, overwrite=True)

# Index the documents; assuming `docs_df` has columns `processed_text` (text) and `docno` (document IDs)
index_ref = indexer.index(docs_df["processed_text"], docs_df["docno"])

# Output index information
print('==========================================================================================')
print(index_ref.toString())
```

```
09:26:37.479 [main] ERROR org.terrier.structures.indexing.Indexer -- Could not rename index
java.io.IOException: Rename of index structure file 'c:\Users\Mazen\Desktop\AAST\Feb 2026\Information Retrieval\week
        at org.terrier.structures.IndexUtil.renameIndex(IndexUtil.java:379)
        at org.terrier.structures.indexing.Indexer.index(Indexer.java:388)
        at org.terrier.structures.indexing.Indexer.index(Indexer.java:355)
==========================================================================================
c:\Users\Mazen\Desktop\AAST\Feb 2026\Information Retrieval\week 6\invertedIndex/data.properties
```

Explore the index

An index has several data structures:

- **the CollectionStatistics**- the salient global statistics of the index. Contains metrics such as the total number of documents, total number of terms, and overall term frequencies.
- **the Lexicon** - the vocabulary of the index, including statistics of the terms, and a pointer into the inverted index. It serves as the vocabulary of the index. It contains unique terms extracted from the collection, stores statistics for each term (e.g., document frequency, collection frequency) and maintains pointers (or offsets) to the corresponding posting list in the inverted index.
- **the inverted index (a PostingIndex**) - contains the posting list for each term, detailing the frequency in which aterm appears in that document. It contains posting lists for each term. A posting list is essentially a list of documents in which the term appears. Each entry in a posting list typically includes the document identifier and the frequency (or positions) of the term in that document.
- **the DocumentIndex** - Stores metadata such as the length of each document and field-specific lengths if the document has multiple fields (e.g., title, body).
- **the MetaIndex** - contains document metadata, such as the docno, and optionally the raw text and the URL of each document.
- **the direct index (also a PostingIndex)** - contains a posting list for each document, detailing which terms occuring in that document and which frequency. The presence of the direct index depends on the Indexing Type that has been applied - single-pass and some memory indices do not provide a direct index.

Let's check the files the index files created.

We can export our index into our machine as follows:

Let's check the statistics about the index we created.

```
print(index_ref.toString())

#we will first load the index
index = pt.IndexFactory.of(index_ref)

#we will call getCollectionStatistics() to check the stats
print(index.getCollectionStatistics())
```

```
c:\Users\Mazen\Desktop\AAST\Feb 2026\Information Retrieval\week 6\invertedIndex/data.properties
Number of documents: 4
Number of terms: 8
Number of postings: 14
Number of fields: 0
Number of tokens: 14
Field names: []
Positions:   false
```

We can check the lexicon which is the **vocabulary** of the collection.

- Nt is the number of unique documents that each term occurs in.
- TF is the total number of occurrences – some weighting models use this instead of Nt.
- The numbers in the @{} are a pointer – they tell Terrier where the postings are for that term in the inverted index data structure.

```
for kv in index.getLexicon():
  print("%s -> %s " % (kv.getKey(), kv.getValue().toString()))
```

```
approach -> term5 Nt=1 TF=1 maxTF=1 @{0 0 0}
breakthrough -> term1 Nt=1 TF=1 maxTF=1 @{0 0 4}
drug -> term0 Nt=2 TF=2 maxTF=1 @{0 0 6}
hope -> term7 Nt=1 TF=1 maxTF=1 @{0 1 2}
new -> term3 Nt=3 TF=3 maxTF=1 @{0 2 0}
patient -> term6 Nt=1 TF=1 maxTF=1 @{0 3 0}
schizophrenia -> term2 Nt=4 TF=4 maxTF=1 @{0 3 6}
treatment -> term4 Nt=1 TF=1 maxTF=1 @{0 4 6}
```

we can also lookup a term in PyTerrier's lexicon:

```
index.getLexicon()["schizophrenia"].toString()
```

```
'term2 Nt=4 TF=4 maxTF=1 @{0 3 6}'
```

```
index.getLexicon()["schizophrenia"]
```

```
<org.terrier.structures.LexiconEntry at 0x1d0b23da450 jclass=org/terrier/structures/LexiconEntry jself=<LocalRef
obj=0x2c033c42 at 0x1d0b223bb90>>
```

The inverted index tells us in which documents each term occurs in. The LexiconEntry is the pointer that tell us where to find the postings for that term in the inverted index.

Let's look in which documents the word "schizophrenia" occurs and its frequency in each document.

Note: we need to preprocess each search term with the same preprocessing steps we performed on the collection.

```
term="schizophrenia"

try:
 pointer = index.getLexicon()[term]
 for posting in index.getInvertedIndex().getPostings(pointer):
    print(posting.toString() + " doclen=%d" % posting.getDocumentLength())
except:
    print("term %s not found"%term)
```

```
ID(0) TF(1) doclen=3
ID(1) TF(1) doclen=3
ID(2) TF(1) doclen=4
ID(3) TF(1) doclen=4
```

How many documents does term "schizophrenia" occur in?

```
index.getLexicon()[term].getDocumentFrequency()
```

```
4
```

What terms occur in the 4th document?

```
di = index.getDirectIndex()
doi = index.getDocumentIndex()
lex = index.getLexicon()
docid = 2 #docids are 0-based #note: postings will be null if the document is empty
for posting in di.getPostings(doi.getDocumentEntry(docid)):
    termid = posting.getId()
    lee = lex.getLexiconEntry(termid)
    print("%s with frequency %d" % (lee.getKey(),posting.getFrequency()))
```

```
schizophrenia with frequency 1
new with frequency 1
treatment with frequency 1
approach with frequency 1
```

```
# Define a function to retrieve document IDs for a given term
def get_document_ids_for_term(index, term):
    try:
        pointer = index.getLexicon()[term]
        doc_ids = [posting.getId() for posting in index.getInvertedIndex().getPostings(pointer)]
        return doc_ids
    except Exception:
        print("Term '%s' not found" % term)
        return []


# -------- Basic AND merge with comparison counting --------
def merge(p1, p2):
    answer = []
    i, j = 0, 0
    comparisons = 0

    while i < len(p1) and j < len(p2):
        comparisons += 1
        if p1[i] == p2[j]:
            answer.append(p1[i])
            i += 1
            j += 1
```

```
        else:
            comparisons += 1
            if p1[i] < p2[j]:
                i += 1
            else:
                j += 1

    return answer, comparisons


# -------- Helper to build skip pointers --------
def add_skip_pointers(postings):
    """
    Returns a structure like:
    [
        {'doc_id': 1, 'next': 1, 'skip': 2},
        {'doc_id': 3, 'next': 2, 'skip': None},
        ...
    ]
    """
    n = len(postings)
    if n == 0:
        return []

    skip_interval = int(n ** 0.5)  # common heuristic: sqrt(n)

    nodes = []
    for i, doc_id in enumerate(postings):
        node = {
            'doc_id': doc_id,
            'next': i + 1 if i + 1 < n else None,
            'skip': i + skip_interval if (skip_interval > 1 and i + skip_interval < n) else None
        }
        nodes.append(node)

    return nodes


# -------- Skip pointer AND merge with comparison counting --------
def skipPointerMerge(p1, p2):
    """
    p1 and p2 are normal sorted posting lists, e.g. [0,1,4,7]
    This function internally adds skip pointers, then applies the algorithm.
    """
    answer = []
    comparisons = 0

    L1 = add_skip_pointers(p1)
    L2 = add_skip_pointers(p2)

    i, j = 0, 0

    while i is not None and j is not None:
        comparisons += 1
        if L1[i]['doc_id'] == L2[j]['doc_id']:
            answer.append(L1[i]['doc_id'])
            i = L1[i]['next']
            j = L2[j]['next']

        else:
            comparisons += 1
            if L1[i]['doc_id'] < L2[j]['doc_id']:
                # Try skip on p1
                while (
                    L1[i]['skip'] is not None
                    and L1[L1[i]['skip']]['doc_id'] <= L2[j]['doc_id']
                ):
                    comparisons += 1
                    i = L1[i]['skip']

                if i is not None and L1[i]['doc_id'] < L2[j]['doc_id']:
                    i = L1[i]['next']

            else:
                # Try skip on p2
                while (
                    L2[j]['skip'] is not None
```

```
                    and L2[L2[j]['skip']]['doc_id'] <= L1[i]['doc_id']
                ):
                    comparisons += 1
                    j = L2[j]['skip']

                if j is not None and L2[j]['doc_id'] < L1[i]['doc_id']:
                    j = L2[j]['next']

    return answer, comparisons


# -------- OR / UNION --------
def union_postings(p1, p2):
    answer = []
    i, j = 0, 0

    while i < len(p1) and j < len(p2):
        if p1[i] == p2[j]:
            answer.append(p1[i])
            i += 1
            j += 1
        elif p1[i] < p2[j]:
            answer.append(p1[i])
            i += 1
        else:
            answer.append(p2[j])
            j += 1

    while i < len(p1):
        answer.append(p1[i])
        i += 1

    while j < len(p2):
        answer.append(p2[j])
        j += 1

    return answer


# -------- NOT --------
def not_postings(p, all_doc_ids):
    """
    Returns all docs that are NOT in p
    p must be sorted
    all_doc_ids must be sorted
    """
    answer = []
    i, j = 0, 0

    while i < len(all_doc_ids) and j < len(p):
        if all_doc_ids[i] == p[j]:
            i += 1
            j += 1
        elif all_doc_ids[i] < p[j]:
            answer.append(all_doc_ids[i])
            i += 1
        else:
            j += 1

    while i < len(all_doc_ids):
        answer.append(all_doc_ids[i])
        i += 1

    return answer


# -------- Edit Distance --------
def edit_distance(s1, s2):
    s1 = str(s1)
    s2 = str(s2)

    m = [[0] * (len(s2) + 1) for _ in range(len(s1) + 1)]

    for i in range(1, len(s1) + 1):
        m[i][0] = i

    for j in range(1, len(s2) + 1):
```

```
        m[0][j] = j

    for i in range(1, len(s1) + 1):
        for j in range(1, len(s2) + 1):
            substitution_cost = 0 if s1[i - 1] == s2[j - 1] else 1

            m[i][j] = min(
                m[i - 1][j - 1] + substitution_cost,  # substitution / match
                m[i - 1][j] + 1,                      # deletion
                m[i][j - 1] + 1                       # insertion
            )

    return m[len(s1)][len(s2)]


# -------- Build vocabulary from index lexicon --------
def get_vocabulary(index):
    return [kv.getKey() for kv in index.getLexicon()]


# -------- Spelling correction --------
def correct_spelling(term, vocabulary):
    if term in vocabulary:
        return term

    best_term = None
    best_distance = float('inf')

    for vocab_term in vocabulary:
        dist = edit_distance(term, vocab_term)
        if dist < best_distance:
            best_distance = dist
            best_term = vocab_term

    print(f"'{term}' not found. Using suggested term: '{best_term}'")
    return best_term
```

```
# Define your query
query = "drug AND schizophrnia"   # intentionally misspelled example

# Split query
terms = query.split(' AND ')

# Build vocabulary once
vocabulary = get_vocabulary(index)

# Correct spelling before retrieving postings
corrected_terms = [correct_spelling(term, vocabulary) for term in terms]

# Retrieve document IDs for each corrected term
postings_lists = {term: get_document_ids_for_term(index, term) for term in corrected_terms}

# Check the terms and calculate the intersection
if len(corrected_terms) < 2:
    print("Not enough terms to perform a merge.")
else:
    result_normal = postings_lists[corrected_terms[0]]
    total_comparisons_normal = 0

    result_skip = postings_lists[corrected_terms[0]]
    total_comparisons_skip = 0

    for i in range(1, len(corrected_terms)):
        operand = postings_lists[corrected_terms[i]]

        result_normal, comps_normal = merge(result_normal, operand)
        total_comparisons_normal += comps_normal

        result_skip, comps_skip = skipPointerMerge(result_skip, operand)
        total_comparisons_skip += comps_skip

    print("Corrected query terms:", corrected_terms)
    print("Documents matching the AND query using normal merge:", result_normal)
    print("Normal merge comparisons:", total_comparisons_normal)
```

```
    print("Documents matching the AND query using skip pointers:", result_skip)
    print("Skip pointer merge comparisons:", total_comparisons_skip)
```

```
'schizophrnia' not found. Using suggested term: 'schizophrenia'
Corrected query terms: ['drug', 'schizophrenia']
Documents matching the AND query using normal merge: [0, 1]
Normal merge comparisons: 2
Documents matching the AND query using skip pointers: [0, 1]
Skip pointer merge comparisons: 2
```

```
term1 = correct_spelling("drug", get_vocabulary(index))
term2 = correct_spelling("schizophrenia", get_vocabulary(index))

p1 = get_document_ids_for_term(index, term1)
p2 = get_document_ids_for_term(index, term2)

all_doc_ids = list(range(len(docs_df)))   # since your doc IDs are 0,1,2,3,...

print("Postings for", term1, ":", p1)
print("Postings for", term2, ":", p2)

print("UNION:", union_postings(p1, p2))
print("NOT", term1, ":", not_postings(p1, all_doc_ids))
```

```
Postings for drug : [0, 1]
Postings for schizophrenia : [0, 1, 2, 3]
UNION: [0, 1, 2, 3]
NOT drug : [2, 3]
```